

WIPRO NGA SELENIUM WITH PYTHON

APRIL 2026 | BATCH 2

CAPSTONE PROJECT

PROJECT CASE STUDY – E-Commerce

PROJECT WEBSITE – Nykaa

MODULE NAME – Men Category

SUPERSET ID – 5370346

Submitted by: Yoga Lakshmi N

Submitted to: Wipro Limited



ABSTRACT

This project focuses on developing an automated testing framework using Selenium WebDriver and Behavior Driven Development (BDD) to validate the functionality of a web application. The framework is designed to test key features such as navigation, product search, filtering, cart operations, shipping details, and complete end-to-end user flow.

The automation is implemented using Python, with Pytest for Selenium-based test execution and Behave for BDD scenarios written in Gherkin language. The framework follows the Page Object Model (POM) design pattern to ensure better code reusability, maintainability, and scalability.

Additional utilities such as logging, screenshot capture, and data handling using Excel/CSV are integrated to enhance test execution and debugging. Allure reporting is used to generate detailed test reports, providing clear insights into test results, failures, and execution flow.

Overall, this project aims to build a robust, efficient, and maintainable automation framework that ensures application quality and supports future enhancements and continuous testing practices.

TABLE OF CONTENTS

S. No	Title	Page No
1	Project Description	1
2	Test Strategy	1
2.1	Selenium-Based Automation Approach	1
2.1.1	Parameterization for Reusability	1
2.1.2	Data-Driven Testing using Excel	1
2.1.3	Utility Layer Implementation	1
2.1.4	Allure Reporting Integration	2
2.2	BDD (Behaviour Driven Development) Approach	2
2.2.1	Gherkin-Based Feature Design	2
2.2.2	Scenario Outline with Examples	2
2.2.3	Data Tables for Structured Inputs	2
2.2.4	CSV-Based Data Handling in BDD	2
2.2.5	Integration with Selenium (Step Definitions)	2
3	Test Environment	3
3.1	Development Tools	3
3.1.1	IDE – PyCharm	3
3.1.2	Version Control – Git	3
3.2	Browser and Web Automation	3
3.2.1	Web Browser – Chrome	3
3.2.2	Automation Tool – Selenium WebDriver	3
3.2.3	Driver Management – WebDriver Manager	3
3.3	Testing Frameworks	3
3.3.1	Pytest	3
3.3.2	Behave (BDD Framework)	3
3.3.3	Gherkin	4
3.4	Data Handling and Configuration	4
3.4.1	OpenPyXL	4

3.4.2	ConfigParser	4
3.5	Reporting and Logging	4
3.5.1	Allure Reports (allure-behave)	4
3.6	Folder Structure	5
3.6.1	BDD	5
3.6.2	Selenium	6
3.7	Framework Folder Structure Explanation	8
3.7.1	Test Scenarios and Test Scripts	8
3.7.2	Page Object Model (Pages)	8
3.7.3	Utilities	8
3.7.4	Configuration Management	8
3.7.5	Test Data Management	9
3.7.6	Reports and Screenshots	9
3.7.7	Logs	9
3.7.8	Execution and Setup Files	9
4	Test Execution	9
4.1	Test Cases	10
4.1.1	BDD Test Cases	10
4.1.2	Selenium Test Cases	11
4.2	Issues Identified	12
4.2.1	OTP Login Limitation	12
4.3	Execution and Reports	12
4.4	Execution Commands	13
4.4.1	Selenium (Pytest Execution)	13
4.4.2	BDD (Behave Execution)	13
5	Test Output	13
5.1	BDD	13
5.1.1	Sample Output Screenshots	13
5.1.2	Logs	16
5.1.3	Allure Reports	18

5.2	Selenium	18
5.2.1	Sample Output Screenshots	18
5.2.2	Logs	22
5.2.3	Allure Reports	23
6	Test Conclusion	23
6.1	Execution Summary	24
6.2	Accuracy Calculation	24
6.3	Observations	24
6.4	Failure Summary	24
6.5	Final Conclusion	24

1. PROJECT DESCRIPTION:

This project is an end-to-end test automation framework developed for the Nykaa e-commerce platform using Python, Selenium WebDriver, Pytest, and Behave (BDD). It follows the Page Object Model (POM) design pattern to ensure modularity, reusability, and maintainability of test scripts, while the BDD approach enables writing human-readable test scenarios using Gherkin syntax for better collaboration between technical and non-technical stakeholders.

The framework supports data-driven testing using Excel, allowing execution of multiple test scenarios such as login, product filtering, cart operations, and checkout flows. Advanced features like parameterization, structured logging, exception handling, and screenshot capture with Allure reporting are integrated to provide detailed test execution insights.

It is designed to handle both positive and negative test scenarios, including edge cases like invalid OTP and incorrect pincode validation during checkout. Special focus has been given to building dynamic and reusable methods, ensuring that the same functions can support multiple test outcomes without breaking end-to-end flows.

By combining BDD for behavior validation and Pytest for execution and assertions, the framework delivers a robust, scalable automation solution that simulates real user behavior. This makes it highly suitable for real-world QA environments and continuous integration pipelines, improving test reliability and maintainability.

2. TEST STRATEGY:

2.1 SELENIUM-BASED AUTOMATION APPROACH:

2.1.1 Parameterization for Reusability

In the Selenium layer, most of the user actions such as clicking categories, applying filters, and selecting options were implemented using parameterization. This helped avoid hardcoding values and allowed the same methods to work for different inputs. As a result, the scripts became more flexible and reusable across multiple test scenarios without duplication.

2.1.2 Data-Driven Testing using Excel

For handling user input fields like shipping details and form data, external data sources such as Excel files were used. This separation of test data from test logic made it easier to run multiple test variations without modifying the code. It also improved maintainability and allowed quick updates to test inputs.

2.1.3 Utility Layer Implementation

A separate utilities layer was created to handle common functionalities such as logging, screenshot capture, and wait handling. These utilities ensured consistent behavior across all test cases and reduced code repetition. Proper exception handling was also implemented to manage runtime issues effectively.

2.1.4 Allure Reporting Integration

Allure reporting was directly integrated into the test scripts to provide detailed execution insights. Screenshots were captured at important steps and attached to the reports, along with logs. This made debugging easier and provided a clear understanding of test execution flow and failures.

2.2 BDD (BEHAVIOUR DRIVEN DEVELOPMENT) APPROACH:

2.2.1 Gherkin-Based Feature Design

BDD was implemented using Gherkin syntax to define test scenarios in a more readable and structured format. The feature files describe user actions in a business-readable way, making it easier for both technical and non-technical stakeholders to understand the test coverage.

2.2.2 Scenario Outline with Examples

For scenarios that required multiple input combinations, Scenario Outline with Examples was used. This allowed the same scenario to run with different data sets, improving coverage without increasing the number of feature files. It also kept the test cases clean and organized.

2.2.3 Data Tables for Structured Inputs

In cases where multiple inputs needed to be passed within a step, data tables were used. This was particularly useful for actions like selecting filters or payment methods. It made the steps more dynamic and avoided writing separate steps for each variation.

2.2.4 CSV-Based Data Handling in BDD

For handling larger datasets, CSV files were integrated with BDD scenarios. This approach helped in managing bulk test data efficiently while keeping the feature files simple. It also ensured consistency between test data and execution.

2.2.5 Integration with Selenium (Step Definitions)

The step definitions acted as a bridge between the Gherkin steps and Selenium page methods. Each step was mapped to reusable functions from the Page Object Model, ensuring a clear separation between test logic and implementation. This made the framework modular and easy to maintain.

3. TEST ENVIRONMENT:

3.1 DEVELOPMENT TOOLS:

3.1.1 IDE – PyCharm

The automation framework was developed using PyCharm, which provided a robust environment for writing and managing Python scripts. Its features like intelligent code suggestions, debugging support, and project structure management helped streamline development and improve productivity.

3.1.2 Version Control – Git

Git was used for version control to manage code changes and maintain different versions of the project. It enabled proper tracking of updates, collaboration, and rollback of changes when required, ensuring code stability throughout development.

3.2 BROWSER AND WEB AUTOMATION:

3.2.1 Web Browser – Chrome

Google Chrome was used as the primary browser for test execution. It provides good compatibility with Selenium and reflects real user behavior, making it suitable for end-to-end testing of the application.

3.2.2 Automation Tool – Selenium WebDriver

Selenium WebDriver was used to automate browser interactions such as clicking elements, entering data, and navigating between pages. It formed the core of the automation framework and enabled simulation of real user workflows.

3.2.3 Driver Management – WebDriver Manager

WebDriver Manager was used to handle browser driver setup automatically. It eliminates the need for manual driver downloads and ensures that the correct driver version is used for the installed browser.

3.3 TESTING FRAMEWORKS:

3.3.1 Pytest

Pytest was used for executing Selenium-based test cases. It provides features like parameterization, fixtures, and easy test execution, which helped in organizing and managing test cases efficiently.

3.3.2 Behave (BDD Framework)

Behave was used to implement Behavior Driven Development (BDD). It allowed writing test scenarios in Gherkin format, making them more readable and aligned with business requirements.

3.3.3 Gherkin

Gherkin syntax was used to define feature files in a structured and human-readable format. It helped describe application behavior using Given-When-Then steps, improving clarity of test scenarios.

3.4 DATA HANDLING AND CONFIGURATION:

3.4.1 OpenPyXL

OpenPyXL was used for reading data from Excel files to support data-driven testing. It enabled the framework to handle multiple test inputs without modifying the core test scripts.

3.4.2 ConfigParser

ConfigParser was used to manage configuration settings such as base URL, browser type, and timeouts. This allowed easy modification of environment settings without changing the code.

3.5 REPORTING AND LOGGING:

3.5.1 Allure Reports (allure-behave)

Allure reporting was integrated using allure-behave to generate detailed and visually rich test reports. It included step-level reporting, screenshots, and logs, which helped in analyzing test results effectively.

Tool / Library	Version	Purpose
Python	3.12.2	Automation scripting
PyCharm	Latest (2026.1)	Code development
Git	2.49.0.windows.1	Version control
Google Chrome	Version 148.0.7778.179	Test execution
Selenium	4.44.0	Browser automation
WebDriver Manager	4.1.1	Driver management
Pytest	9.0.3	Test execution
Behave (BDD)	9.0.3	BDD framework
Gherkin	-	Scenario definition
OpenPyXL	3.1.5	Excel data handling
ConfigParser	Built-in	Configuration management
Allure	2.41.0	Test reporting

3.6 FOLDER STRUCTURE:

3.6.1 BDD:

BDD/

```
|
|
|—— config/
|  └── config.ini
```

```
|
|
|— features/
| |— end_to_end.feature
| |— filter.feature
| |— invalid_pincode.feature
| |— login.feature
| |— search.feature
| |— sort.feature
| |— environment.py
| |— steps/
| |   |— end_to_end_steps.py
| |   |— filter_steps.py
| |   |— invalid_pincode_steps.py
| |   |— login_steps.py
| |   |— search_steps.py
| |   |— sort_steps.py
|
|
|— pages/
| |— base_page.py
| |— cart_page.py
| |— category_page.py
| |— home_page.py
| |— login_page.py
| |— men_page.py
| |— payment_page.py
| |— product_page.py
| |— shipping_details_page.py
|
|
|— utils/
| |— config_reader.py
| |— csv_reader.py
| |— logger.py
| |— screenshot_util.py
|
|
|— testdata/
| |— data.csv
```

```
|
|
|— logs/
|   └─ automation.log
|
|
|— reports/
|   └─ allure-report/
|   └─ allure-results/
|   └─ screenshots/
|
|
|— behave.ini
|— requirements.txt
└─ runtests.py
```

3.6.2 SELENIUM:

Selenium

```
|
|
|— pytest.ini
|— requirements.txt
|— conftest.py
|
|— config
|   └─ config.properties
|
|
|— pages
|   └─ base_page.py
|   └─ cart_page.py
|   └─ home_page.py
|   └─ login_page.py
|   └─ luxe_store_page.py
|   └─ men_page.py
|   └─ payment_page.py
|   └─ product_page.py
|   └─ shipping_details_page.py
|
```

```
|— tests
|   |— test_01_login.py
|   |— test_02_cart_empty.py
|   |— test_03_search.py
|   |— test_04_filters.py
|   |— test_05_cart.py
|   |— test_06_shipping_details.py
|   |— test_07_checkout_navigation.py
|   |— test_08_end_to_end.py
|
|— utils
|   |— excel_reader.py
|   |— logger.py
|   |— screenshot_util.py
|
|— test_data
|   |— data.xlsx
|
|— logs
|   |— automation.log
|
|— reports
|   |— allure-report
|   |— allure-results
|   |— screenshots
```

3.7 FRAMEWORK FOLDER STRUCTURE EXPLANATION (BDD + SELENIUM)

3.7.1 Test Scenarios and Test Scripts

The framework includes both BDD and Selenium-based test implementations.

The features directory contains Gherkin-based feature files that define application behavior in a human-readable format, covering functionalities such as login, search, filtering, sorting, and end-to-end flow. The corresponding step definitions are implemented in the steps folder, where each step is mapped to Python functions.

In parallel, the tests directory contains Pytest-based test scripts, where each file represents a specific functionality. These scripts validate both positive and negative scenarios and allow independent execution of test cases. This dual approach ensures both business-readable scenarios (BDD) and detailed technical validation (Selenium).

3.7.2 Page Object Model (pages)

The pages module follows the Page Object Model (POM) design pattern. Each webpage is represented as a class containing element locators and reusable methods. This structure is shared across both BDD and Selenium frameworks, ensuring consistency in UI interactions such as login, product selection, cart operations, and payment flow. It reduces code duplication and simplifies maintenance when application changes occur.

3.7.3 Utilities

The utils folder contains reusable helper modules used across the framework:

- config_reader.py – Reads configuration values dynamically (BDD)
- csv_reader.py – Handles CSV-based data-driven testing (BDD)
- excel_reader.py – Reads Excel test data (Selenium)
- logger.py – Manages structured logging
- screenshot_util.py – Captures screenshots during execution

These utilities improve reusability, maintain consistency, and support debugging across both frameworks.

3.7.4 Configuration Management

Configuration settings are managed using:

- config.ini (BDD)
- config.properties (Selenium)

These files store details such as base URL, browser type, and timeouts. This separation allows easy updates to execution settings without modifying test scripts, making the framework flexible across environments.

3.7.5 Test Data Management

Test data is maintained using:

- CSV files in the testdata folder (BDD)
- Excel files in the test_data folder (Selenium)

This enables data-driven testing, allowing multiple input combinations to be executed efficiently while validating both positive and negative scenarios.

3.7.6 Reports and Screenshots

The framework integrates Allure reporting for both BDD and Selenium executions.

- allure-results stores raw execution data
- allure-report contains generated HTML reports
- screenshots (inside reports) stores captured images

These reports provide detailed insights into test execution, including step-level results, failures, and visual evidence for debugging.

3.7.7 Logs

Execution logs are maintained in the logs folder (automation.log). Logging captures detailed information about test steps, execution flow, and errors, which helps in debugging and analysis.

3.7.8 Execution and Setup Files

- behave.ini – Configures BDD execution settings
- pytest.ini – Configures Pytest execution
- conftest.py – Contains shared fixtures like browser setup/teardown (Selenium)
- requirements.txt – Lists all dependencies required for the project
- runtests.py – Acts as the entry point to execute BDD test scenarios

4. TEST EXECUTION:

4.1 TEST CASES

4.1.1 BDD Test Cases

TC_01 – Complete Navigation Flow

Steps: Navigate from home page to payment page

Expected: User should be able to navigate across pages smoothly

Result: Pass

TC_02 – Apply Filters (Valid Data)

Steps: Apply Brand filter = Dove, Category = Hair Care

Expected: Filtered products matching “Dove” should be displayed

Result: Pass

TC_03 – Apply Filters (Invalid Data)

Steps: Apply Brand filter = InvalidBrand

Expected: No filter text should be displayed

Result: Pass

TC_04 – Invalid Pincode Validation

Steps: Enter invalid pincode in shipping section

Expected: Error message should be displayed

Result: Pass

TC_05 – OTP Page Load

Steps: Enter mobile number and proceed to login

Expected: OTP page should load

Result: Fail (OTP restriction in automation)

TC_06 – Search (Invalid Input)

Steps: Search using invalid input (@@@@)

Expected: Error message should be displayed in the page saying “Thanks for visiting”

Result: Pass

TC_07 – Search (Valid Input)

Steps: Search for product (serums)

Expected: Relevant products should be displayed

Result: Pass

TC_08 – Sorting Functionality

Steps: Apply sorting on product list

Expected: Products should be sorted correctly

Result: Pass

TC_09 – Empty Cart Validation

Steps: Open cart without adding products

Expected: Empty cart message should be displayed

Result: Pass

4.1.2 Selenium Test Cases

TC_01 – Login Test

Steps: Enter mobile number and attempt login

Expected: User should reach OTP page

Result: Fail (OTP restriction in automation)

TC_02 – Empty Cart Test

Steps: Navigate to cart

Expected: Empty cart message should be displayed

Result: Pass

TC_03 – Search Invalid Input

Steps: Search using special characters (@\$%^)

Expected: No valid results should be shown

Result: Pass

TC_04 – Filter Test (Valid Brand)

Steps: Apply Brand filter = Gucci

Expected: Filtered products should match brand

Result: Pass

TC_05 – Filter Test (Invalid Brand)

Steps: Apply Brand filter = InvalidBrand

Expected: No results or appropriate handling

Result: Pass

TC_06 – Cart Verification

Steps: Add product to cart

Expected: Product should be visible in cart

Result: Pass

TC_07 – Shipping Details Negative Test

Steps: Enter invalid shipping details

Expected: Error message should be displayed

Result: Pass

TC_08 – Checkout Navigation

Steps: Proceed to checkout

Expected: User should be redirected to checkout page

Result: Pass

TC_09 – End-to-End Test

Steps: Perform full flow from home page to payment page

Expected: Complete flow should execute successfully

Result: Pass

4.2 ISSUES IDENTIFIED

4.2.1 OTP Login Limitation

Description: Automation is unable to handle OTP-based authentication during login.

Impact: Login-related test cases fail due to inability to proceed beyond OTP verification.

Type: External Dependency / Security Restriction

4.3 EXECUTION AND REPORTS

- The framework includes logging and reporting mechanisms to track test execution and analyze results effectively.
- Allure Report is generated to provide a detailed view of test execution, including step-level results and failure analysis.
- Screenshots are captured during test execution, especially for failed scenarios, to assist in debugging.
- Execution logs are maintained in the logs/automation.log file, which records detailed information about test steps, errors, and execution flow.

4.4 EXECUTION COMMANDS

The following commands are used to execute test cases in both Selenium and BDD frameworks:

4.4.1 Selenium (Pytest Execution):

```
pytest -v
```

This command runs all Selenium-based test cases with verbose output, providing detailed execution results for each test.

4.4.2 BDD (Behave Execution):

```
python runtests.py
```

This command triggers the execution of BDD feature files using the Behave framework through the main runner script.

5. TEST OUTPUT:

5.1 BDD:

5.1.1 Sample Output Screenshots:

The test execution process generates extensive logs, screenshots, and reporting data. Including all artifacts would significantly increase the size of the document; therefore, only selected samples are presented here for reference. The complete set of execution outputs, including detailed logs,

screenshots, and Allure reports, is maintained within the project directory and is available in the GitHub repository for further review.

Git repository link:

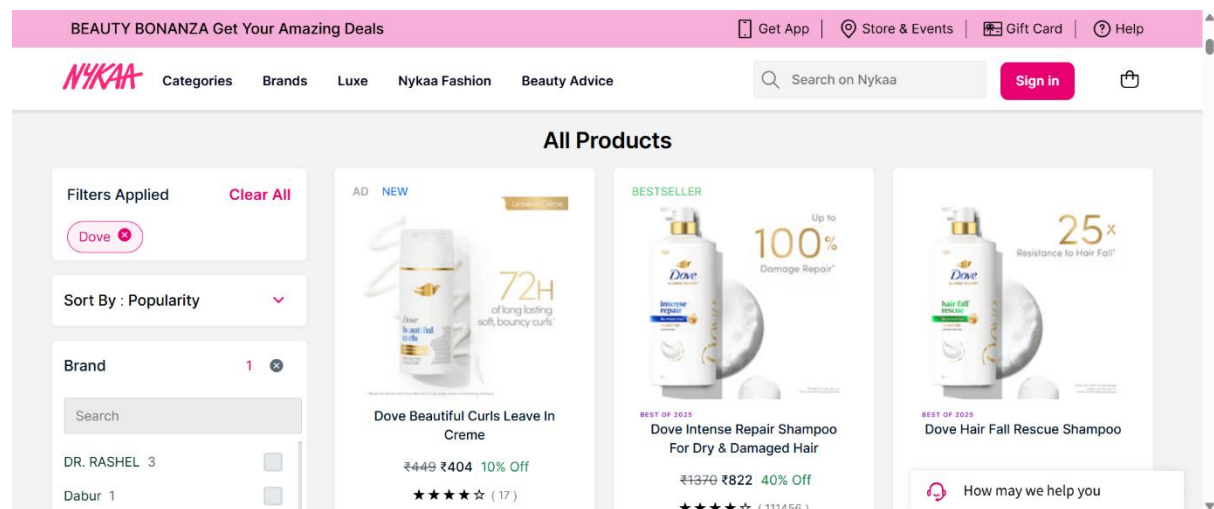
https://github.com/Yogalakshmi85/Yoga_5370346_WiproCapstone.git

Click here to view.

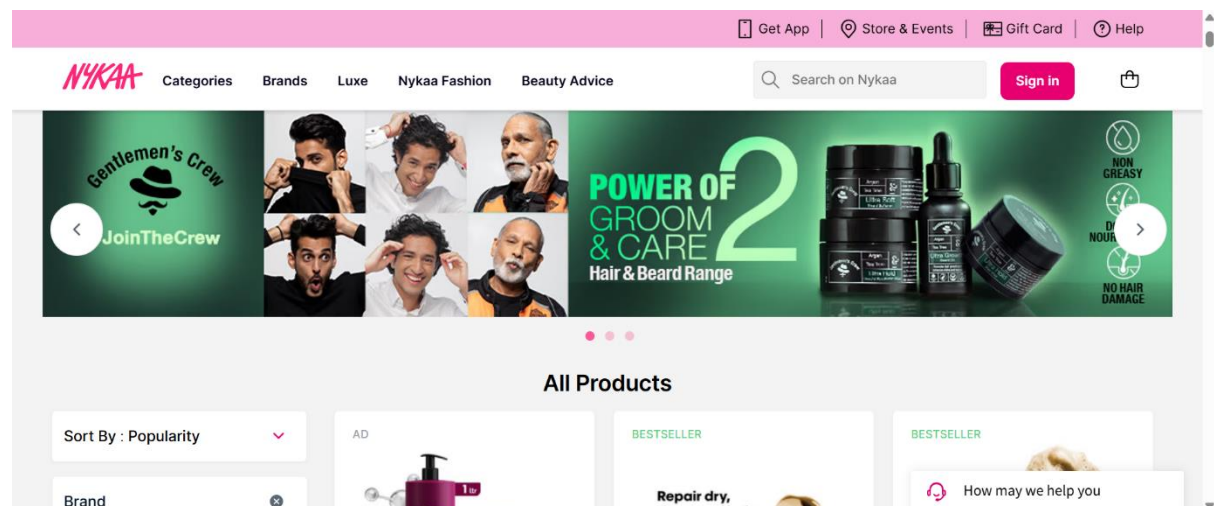
TC_01 – Complete Navigation Flow:

Complete logs and screenshots can be accessed in the project repository.

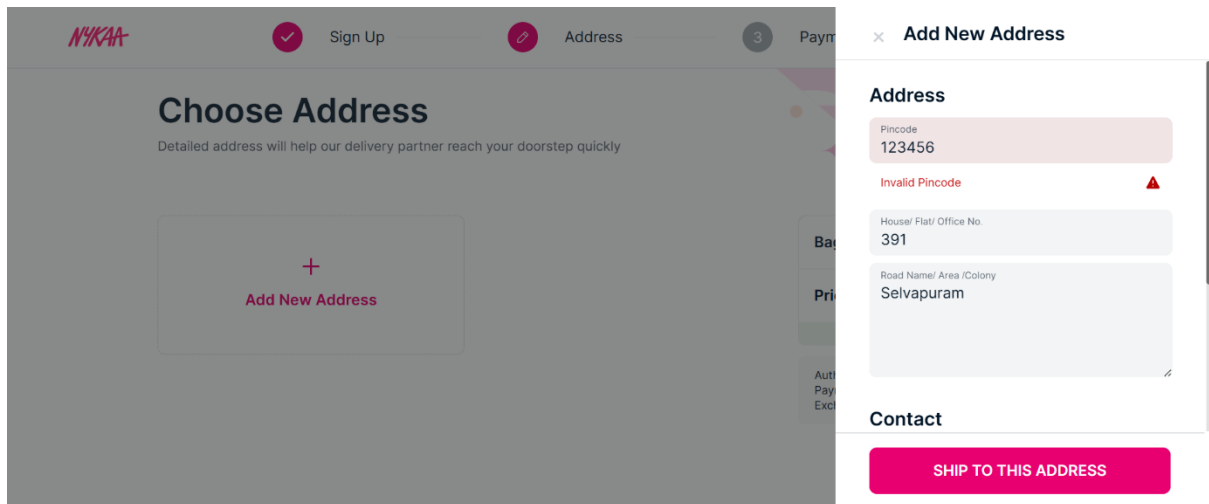
TC_02 – Apply Filters (Valid Data)



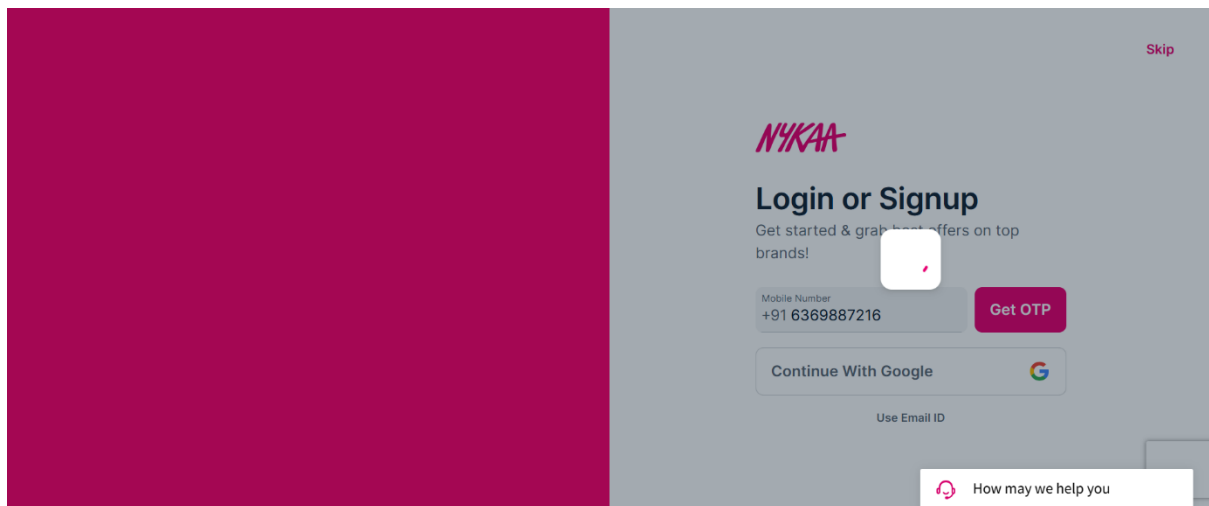
TC_03 – Apply Filters (Invalid Data) – No filters applied.



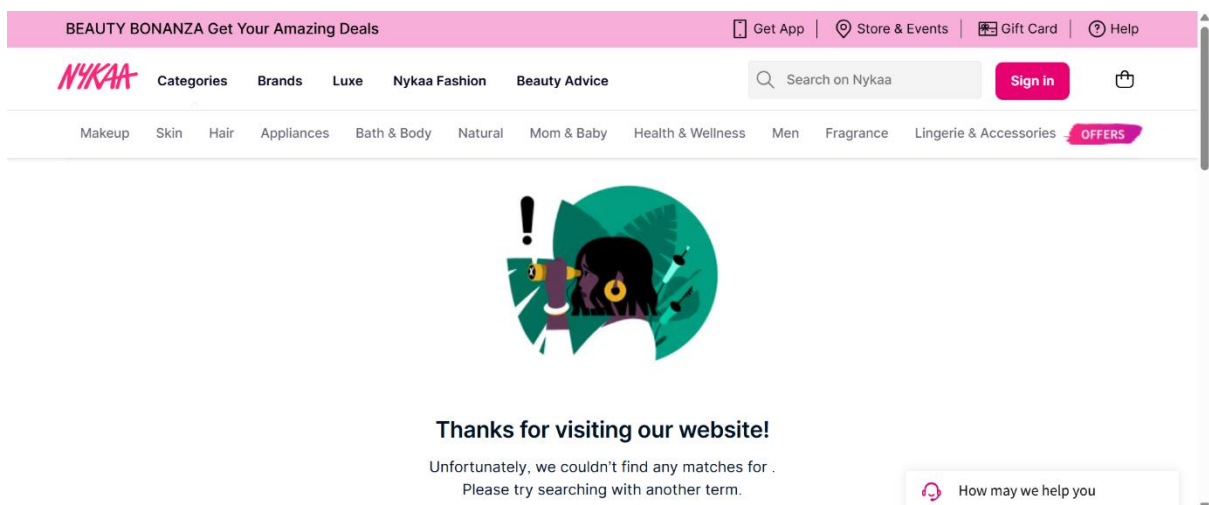
TC_04 – Invalid Pincode Validation



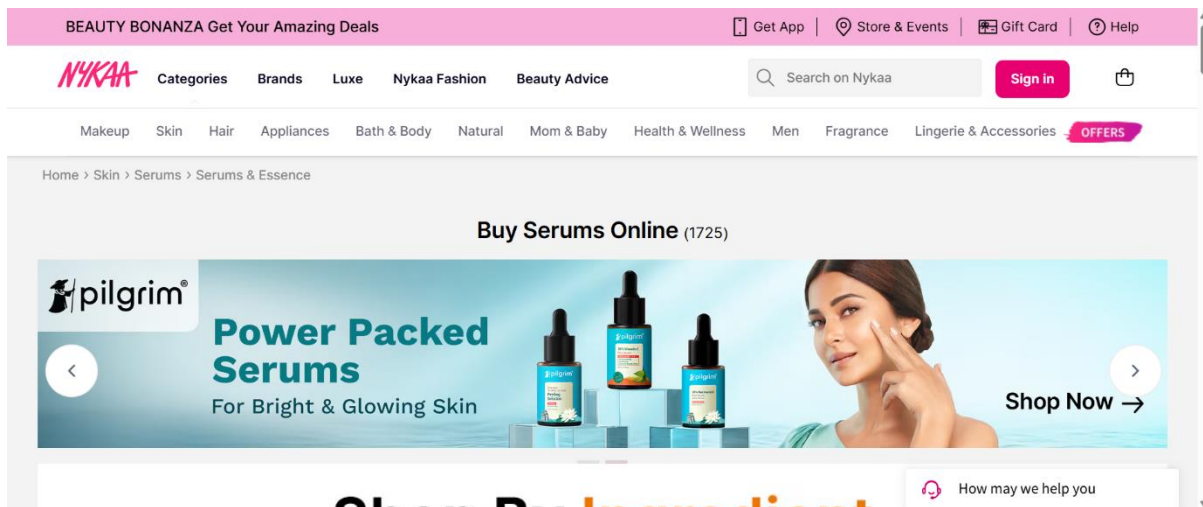
TC_05 – OTP Page Load (AssertionError: OTP verification page did not load)



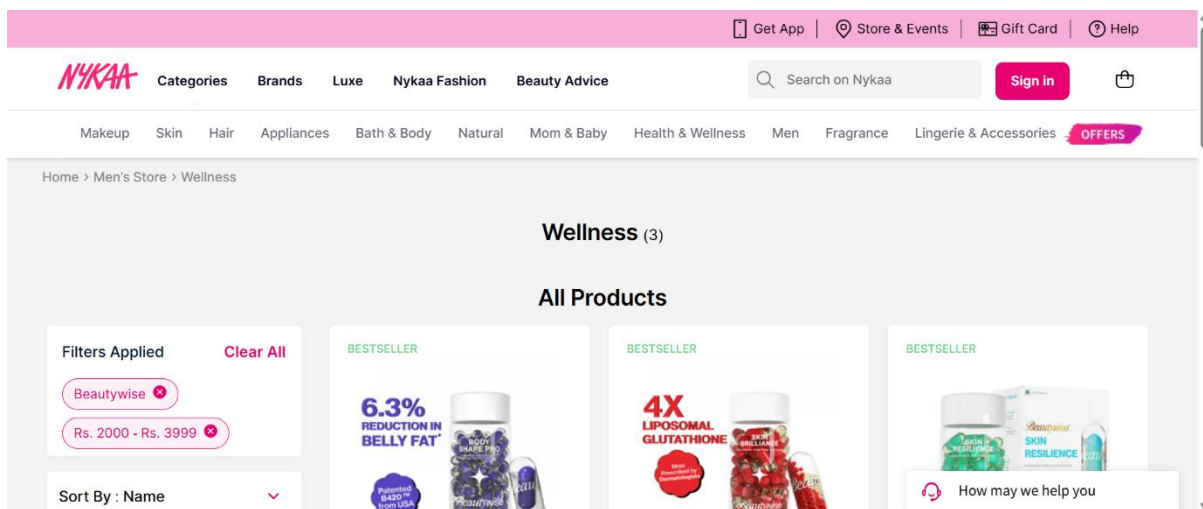
TC_06 – Search (Invalid Input)



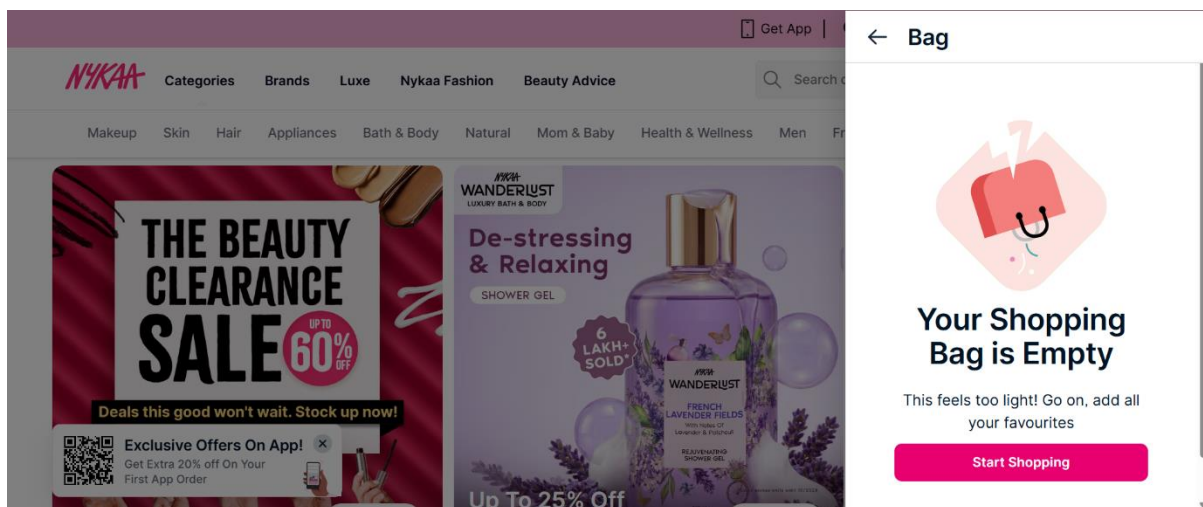
TC_07 – Search (Valid Input)



TC_08 – Sorting Functionality(By Name):



TC_09 – Empty Cart Validation



5.1.2 Logs:

Sample:

2026-05-24 00:57:54,044 - INFO - Opened Nykaa homepage

2026-05-24 00:57:57,785 - INFO - Screenshot saved at:
reports/screenshots/Homepage_Loaded_20260524_005756.png

2026-05-24 00:57:57,809 - INFO - Attempting to navigate to section: men

2026-05-24 00:57:58,522 - INFO - Clicked on men link

2026-05-24 00:57:58,566 - INFO - Switched to new window/tab

2026-05-24 00:58:00,256 - INFO - Assertion passed: men page successfully loaded

2026-05-24 00:58:01,110 - INFO - Navigation successful: men page loaded

2026-05-24 00:58:02,829 - INFO - Screenshot saved at:
reports/screenshots/Section_men_20260524_005801.png

2026-05-24 00:58:02,846 - INFO - Attempting to navigate to health-nutrition: health-nutrition

2026-05-24 00:58:05,834 - INFO - Clicked Category banner for health-nutrition

2026-05-24 00:58:05,939 - INFO - Switched to new window/tab

2026-05-24 00:58:06,625 - INFO - Assertion passed: health-nutrition page successfully loaded

2026-05-24 00:58:06,762 - INFO - Navigation successful: health-nutrition page loaded

2026-05-24 00:58:07,178 - INFO - Screenshot saved at: reports/screenshots/health-nutrition_20260524_005806.png

2026-05-24 00:58:07,354 - INFO - Assertion passed: category page loaded successfully

2026-05-24 00:58:07,504 - INFO - Expanding Brand filter

2026-05-24 00:58:08,804 - INFO - Selecting Brand: Beautywise

2026-05-24 00:58:11,584 - INFO - Applied filter: Brand -> Beautywise

2026-05-24 00:58:11,984 - INFO - Screenshot saved at:
reports/screenshots/Filter_Beautywise_20260524_005811.png

2026-05-24 00:58:11,994 - INFO - Expanding Price filter

2026-05-24 00:58:12,926 - INFO - Selecting Price: Rs. 2000 - Rs. 3999

2026-05-24 00:58:13,661 - INFO - Price filter applied, products visible

2026-05-24 00:58:13,661 - INFO - Applied filter: Price -> Rs. 2000 - Rs. 3999

2026-05-24 00:58:14,023 - INFO - Screenshot saved at: reports/screenshots/Filter_Rs._2000_-
_Rs._3999_20260524_005813.png

2026-05-24 00:58:14,036 - INFO - Assertion passed: Products filtered

2026-05-24 00:58:14,041 - INFO - Selecting first product card

2026-05-24 00:58:15,015 - INFO - Switched to product tab

2026-05-24 00:58:15,016 - INFO - Product selected successfully

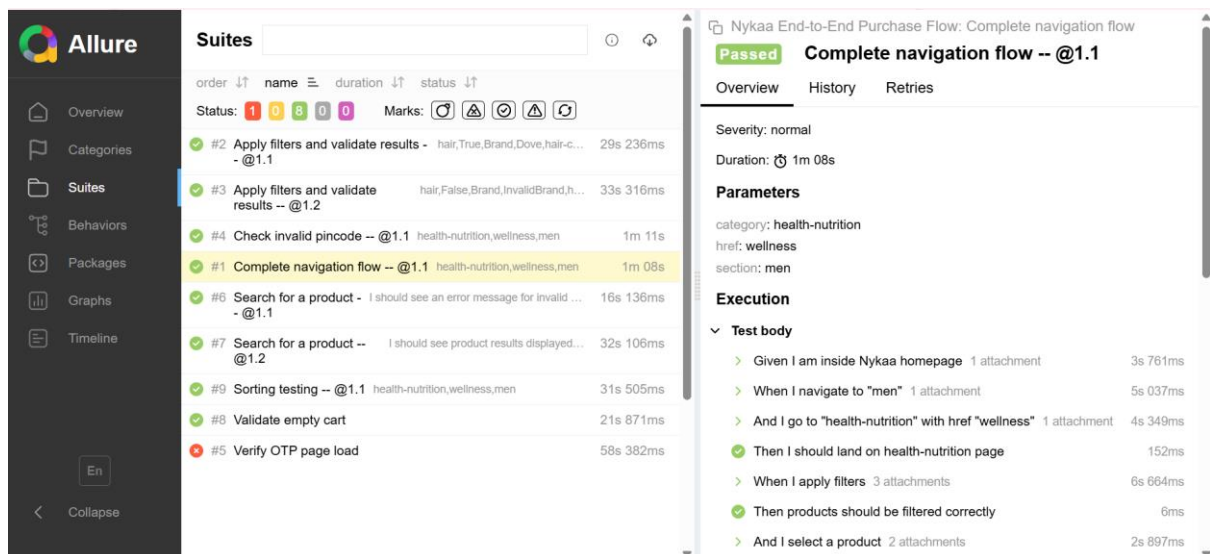
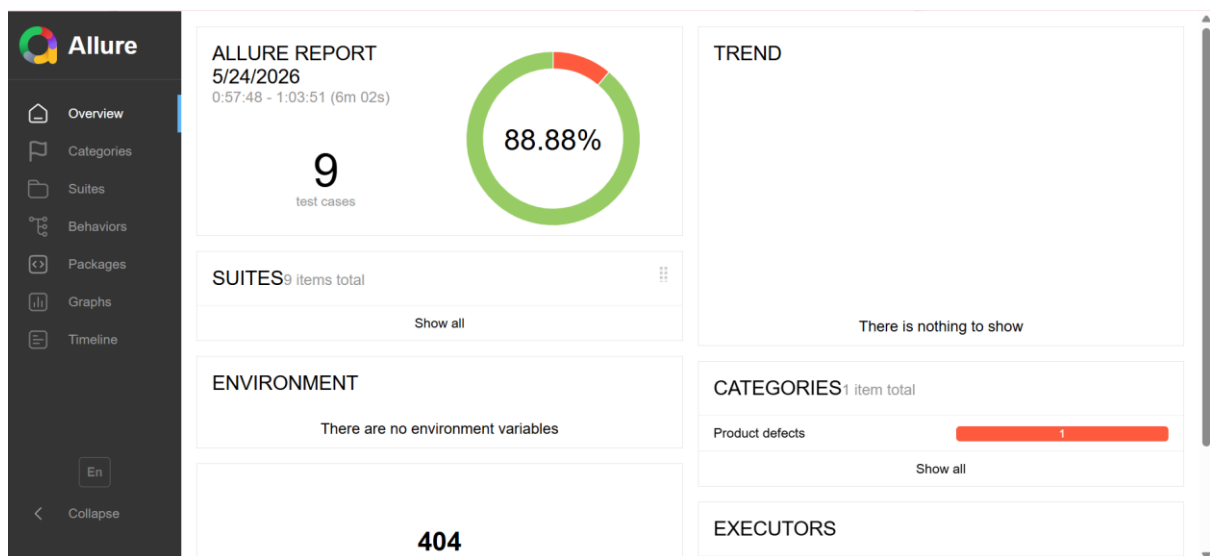
2026-05-24 00:58:16,927 - INFO - Screenshot saved at:
reports/screenshots/Product_Selected_20260524_005815.png

2026-05-24 00:58:16,936 - INFO - Attempting to add product to cart

2026-05-24 00:58:16,937 - INFO - Starting add to cart process

2026-05-24 00:58:21,067 - INFO - Clicked 'Add to Bag' button

5.1.3 Allure Reports:

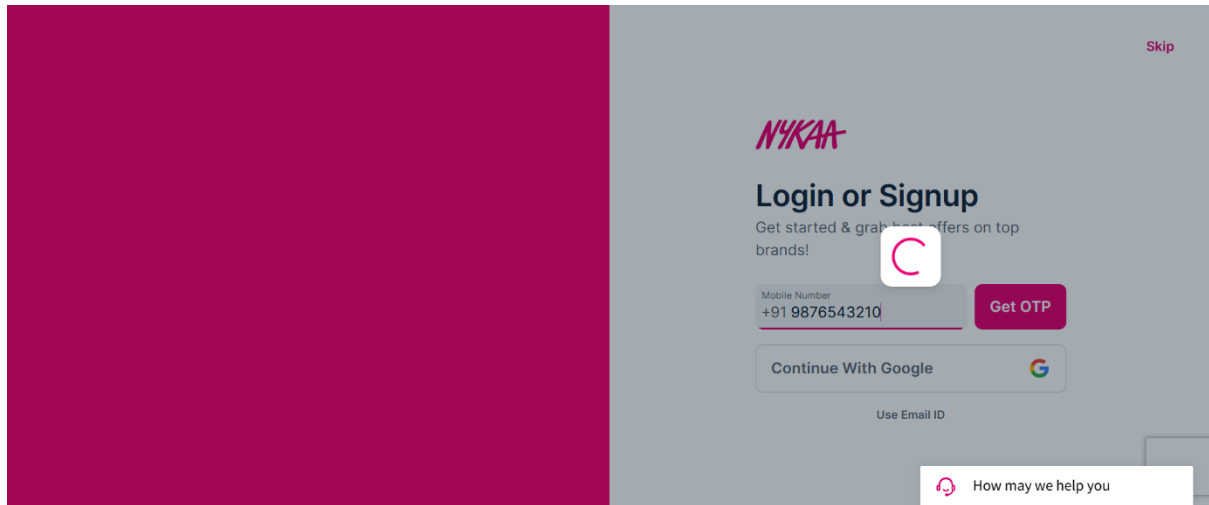


5.2 SELENIUM:

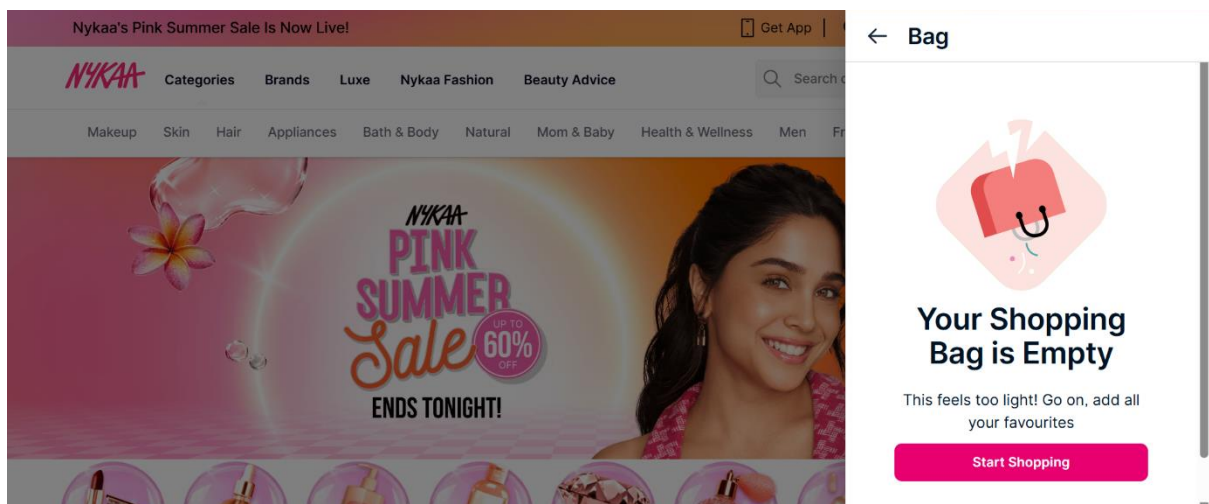
5.2.1 Sample Output Screenshot:

TC_01 – Login Test

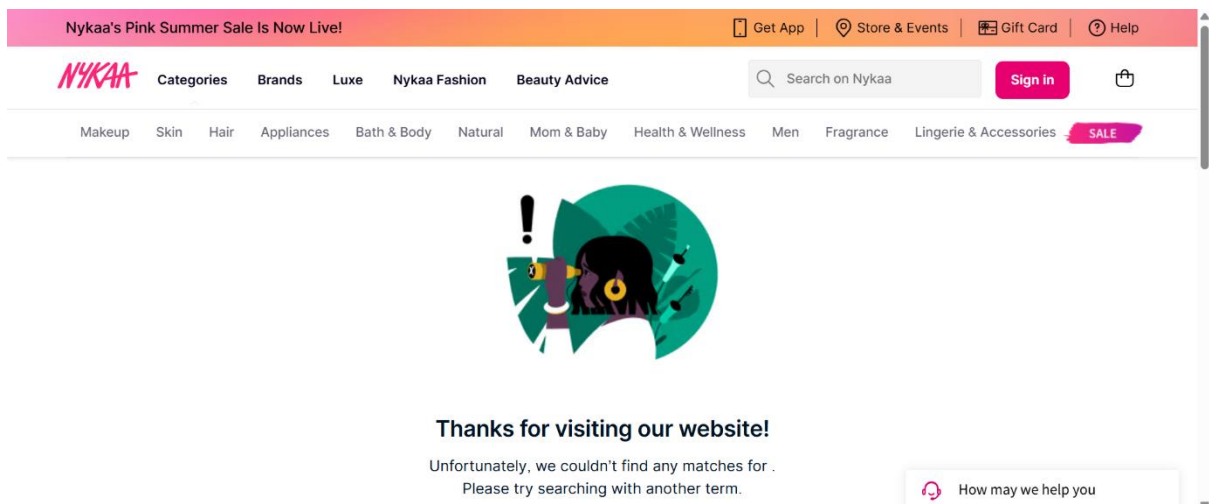
AssertionError: OTP not generated - Nykaa may restrict OTP generation for automation



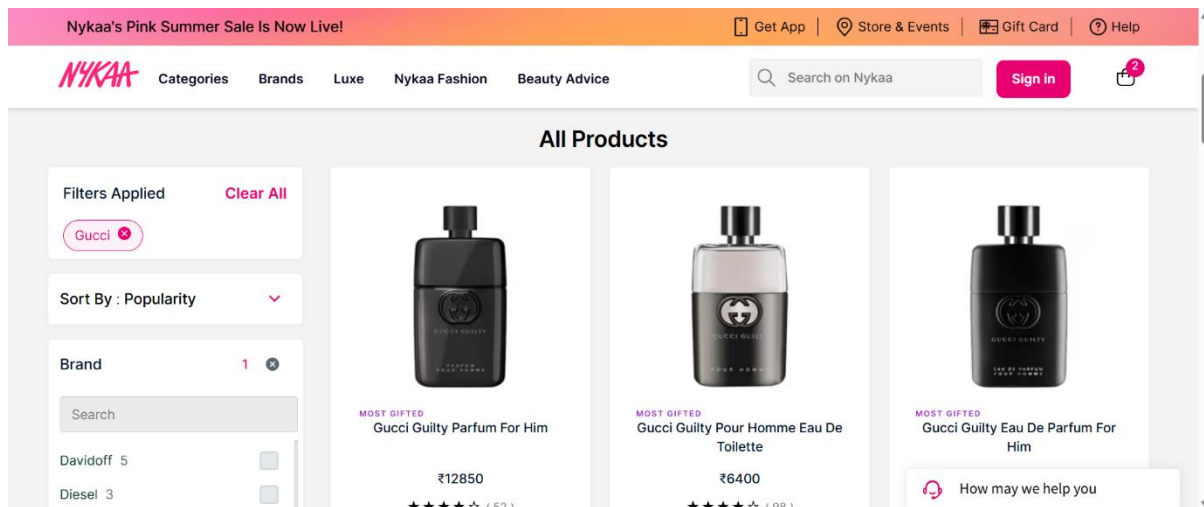
TC_02 – Empty Cart Test



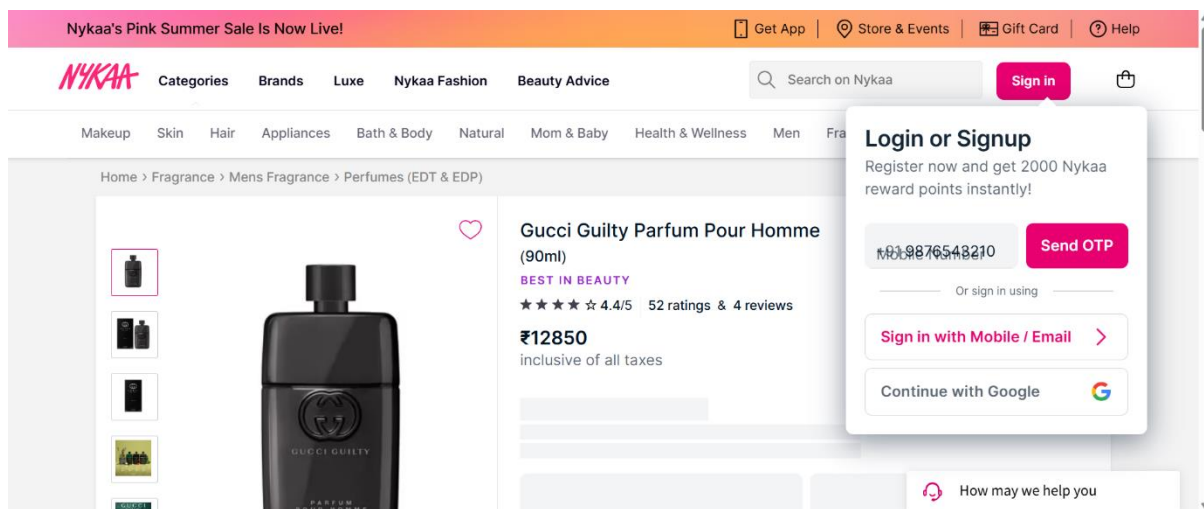
TC_03 – Search Invalid Input



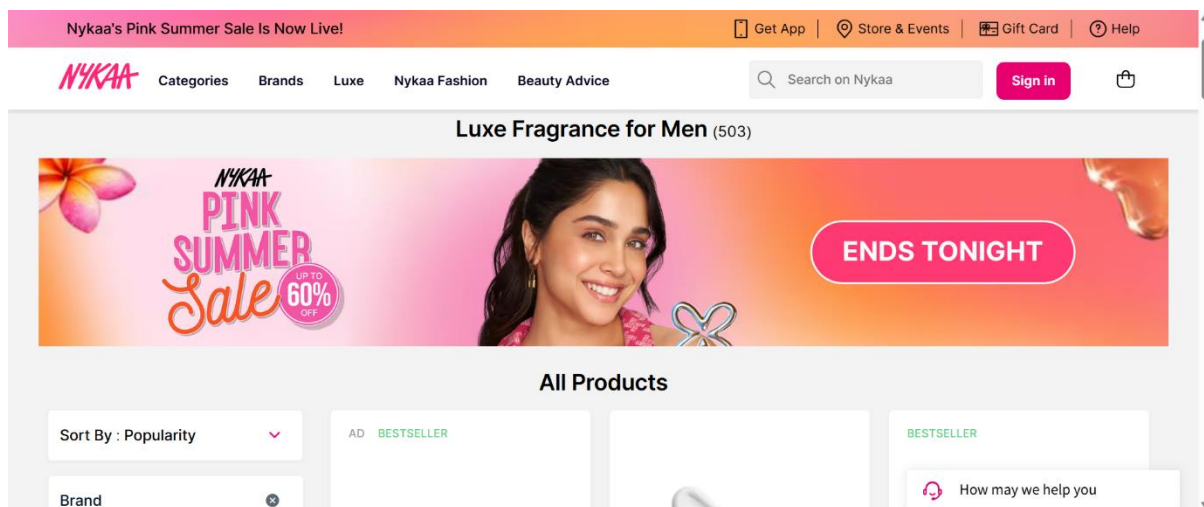
TC_04 – Filter Test (Valid Brand)



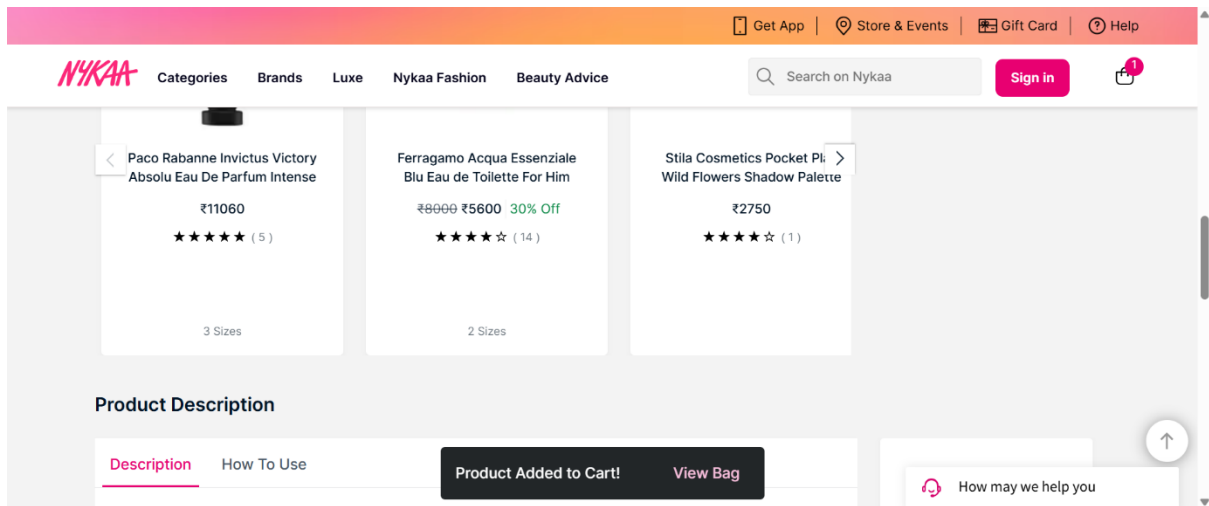
Renders to product



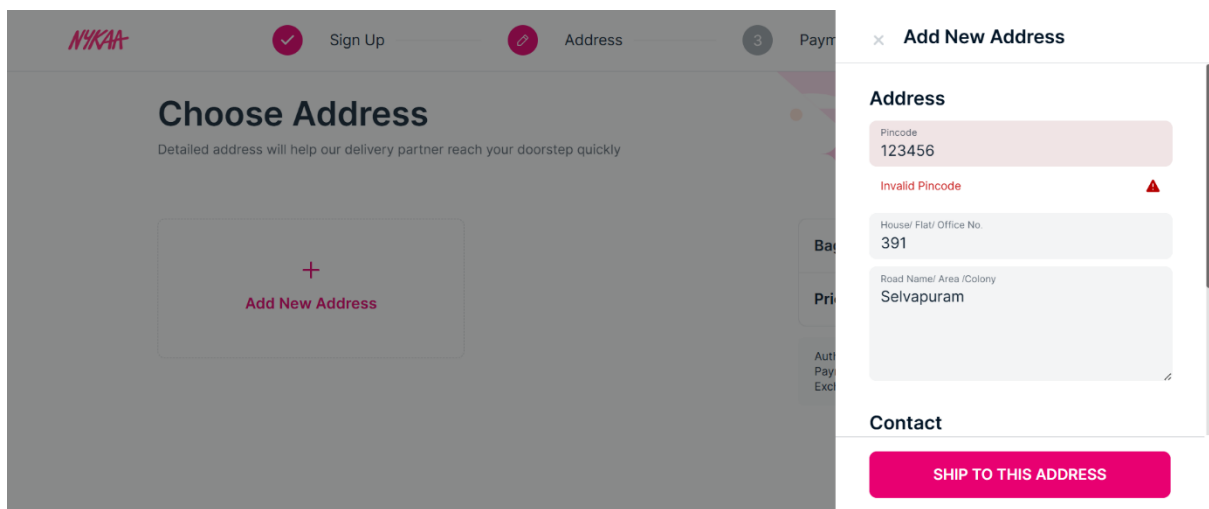
TC_05 – Filter Test (Invalid Brand) – no product selected



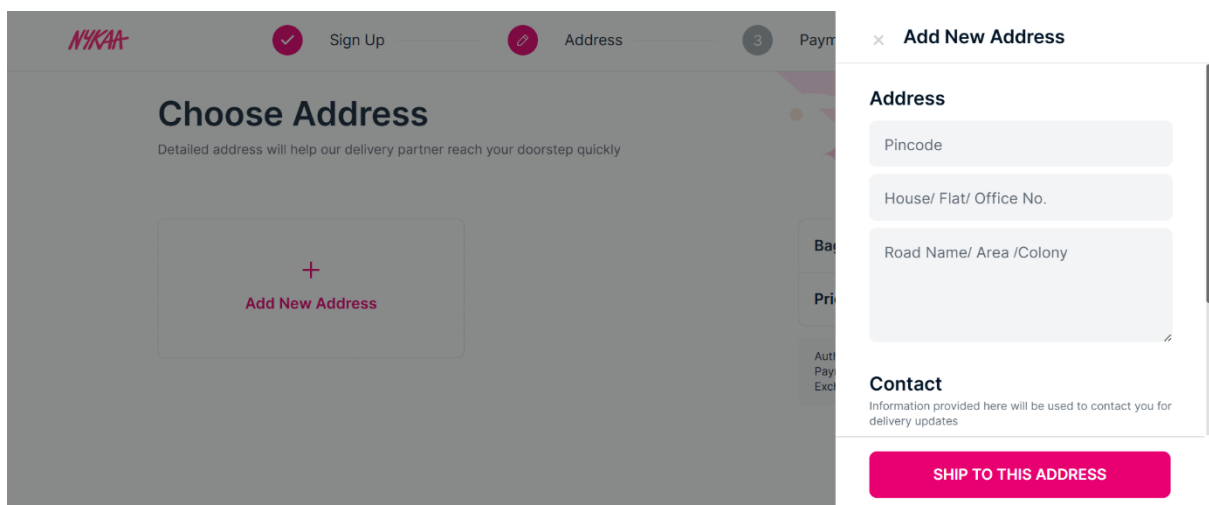
TC_06 – Cart Verification



TC_07 – Shipping Details Negative Test



TC_08 – Checkout Navigation



TC_09 – End-to-End Test

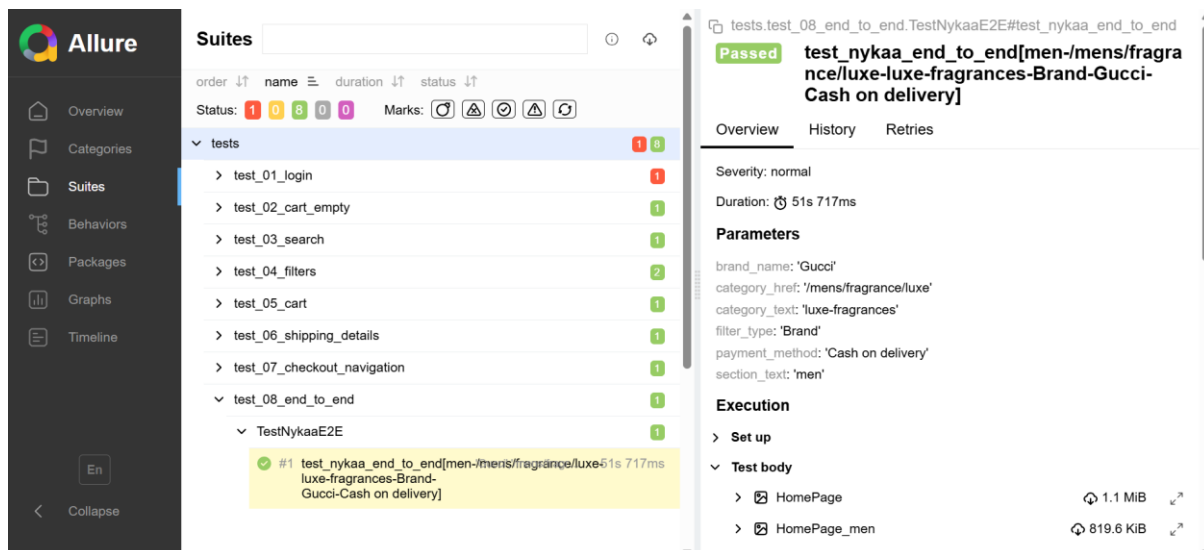
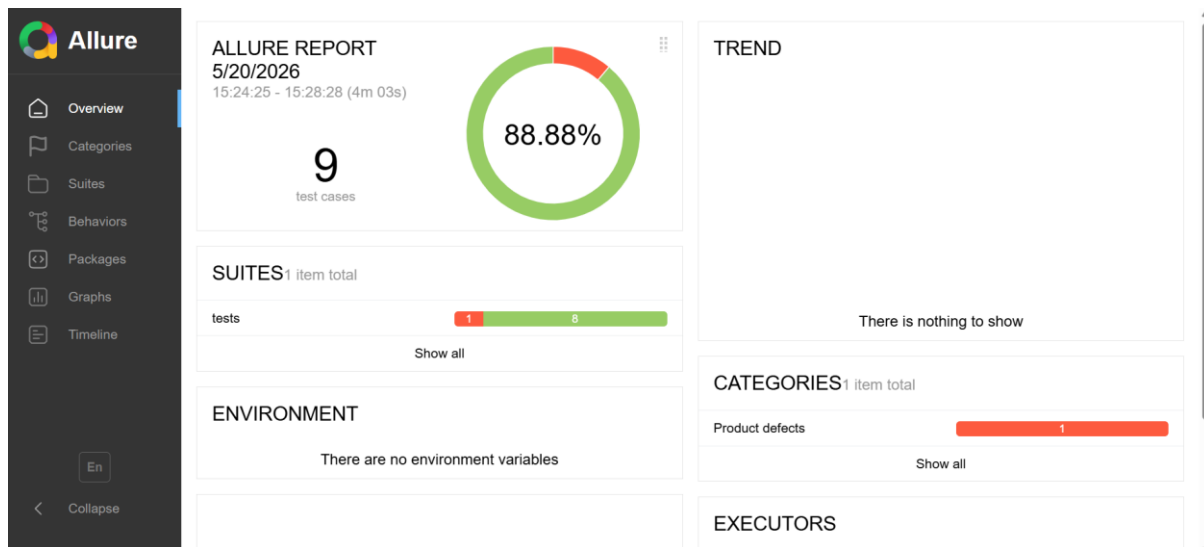
Complete logs and screenshots can be accessed in the project repository.

5.2.2 Logs:

Sample logs:

2026-05-20 15:24:25,119 - INFO - -----
2026-05-20 15:24:25,119 - INFO - Signing in
2026-05-20 15:24:25,120 - INFO - -----
2026-05-20 15:24:25,120 - INFO - Starting sign-in test for brand: 9876543210
2026-05-20 15:24:28,454 - INFO - Clicking Sign in button
2026-05-20 15:24:30,444 - INFO - Entering phone number: 9876543210
2026-05-20 15:24:31,080 - INFO - Clicking Send OTP
2026-05-20 15:24:32,816 - INFO - Navigated to Auth page
2026-05-20 15:24:34,181 - INFO - Clicking Get OTP
2026-05-20 15:24:36,450 - ERROR - OTP verification page did not load, OTP not generated - Nykaa
may restrict OTP generation for automation
2026-05-20 15:24:36,916 - INFO - Screenshot saved at:
reports/screenshots/LoginPage_LoginFailure_20260520_152436.png
2026-05-20 15:24:39,568 - INFO - Opened base URL: <https://www.nykaa.com/>
2026-05-20 15:24:39,569 - INFO - -----
2026-05-20 15:24:39,569 - INFO - STARTING TEST: test_cart_empty
2026-05-20 15:24:39,569 - INFO - -----
2026-05-20 15:24:39,569 - INFO - Validating empty cart
2026-05-20 15:24:41,584 - INFO - Clicked bag icon
2026-05-20 15:24:41,767 - INFO - Assertion passed: Cart is empty
2026-05-20 15:24:42,444 - INFO - Screenshot saved at:
reports/screenshots/HomePage_EmptyCart_20260520_152441.png
2026-05-20 15:24:42,454 - INFO - -----
2026-05-20 15:24:42,455 - INFO - TEST ENDED: Cart is empty as expected
2026-05-20 15:24:42,456 - INFO - -----
2026-05-20 15:24:42,479 - INFO - -----
2026-05-20 15:24:42,479 - INFO - STARTING INVALID SEARCH VERIFICATION
2026-05-20 15:24:42,480 - INFO - -----
2026-05-20 15:24:42,480 - INFO - Starting test for brand: @\$%^
2026-05-20 15:24:43,773 - INFO - Searching for product: @\$%^
2026-05-20 15:24:45,898 - INFO - Screenshot saved at:
reports/screenshots/Search_invalid_keys_20260520_152444.png
2026-05-20 15:24:45,902 - INFO - Entered search text
2026-05-20 15:24:47,131 - INFO - Pressed Enter for search
2026-05-20 15:24:47,181 - INFO - Assertion passed: Error message displayed
2026-05-20 15:24:47,482 - INFO - Screenshot saved at:
reports/screenshots/Search_Error_20260520_152447.png
2026-05-20 15:24:47,489 - INFO - Test passed: Invalid search handled correctly
2026-05-20 15:24:47,490 - INFO - -----
2026-05-20 15:24:47,491 - INFO - ENDING INVALID SEARCH VERIFICATION
2026-05-20 15:24:47,491 - INFO - -----

5.2.3 Allure Reports:



6. TEST CONCLUSION:

The test execution was successfully carried out for both Selenium and BDD frameworks. The automation suite covered major functionalities including navigation, search, filtering, cart operations, shipping, and end-to-end flow.

6.1 EXECUTION SUMMARY

Metric	Count
Total Test Cases	18
Test Cases Passed	16
Test Cases Failed	2
Test Cases Skipped	0
Accuracy	88.89%

6.2 ACCURACY CALCULATION

Accuracy = (Number of Passed Tests / Total Test Cases) × 100

Accuracy = (16 / 18) × 100 = 88.89%

6.3 OBSERVATIONS

- Majority of test cases executed successfully, indicating that the application is stable for core functionalities.
- Failures observed are related to OTP-based authentication, which cannot be automated due to security restrictions.
- No critical functional defects were identified during execution.

6.4 FAILURE SUMMARY

- Login Test (Selenium) – Failed due to OTP restriction
- OTP Page Load (BDD) – Failed due to external authentication limitation

6.5 FINAL CONCLUSION

The automation framework is stable, maintainable, and capable of validating key application workflows effectively. Both Selenium and BDD implementations are functioning as expected, with minor limitations due to external dependencies. The framework can be further enhanced to handle advanced scenarios and integrated into CI/CD pipelines for continuous testing.